



WAYCON internship :

WAYCON Realtime Forensics Pipeline — Final and Full Architecture

Goal

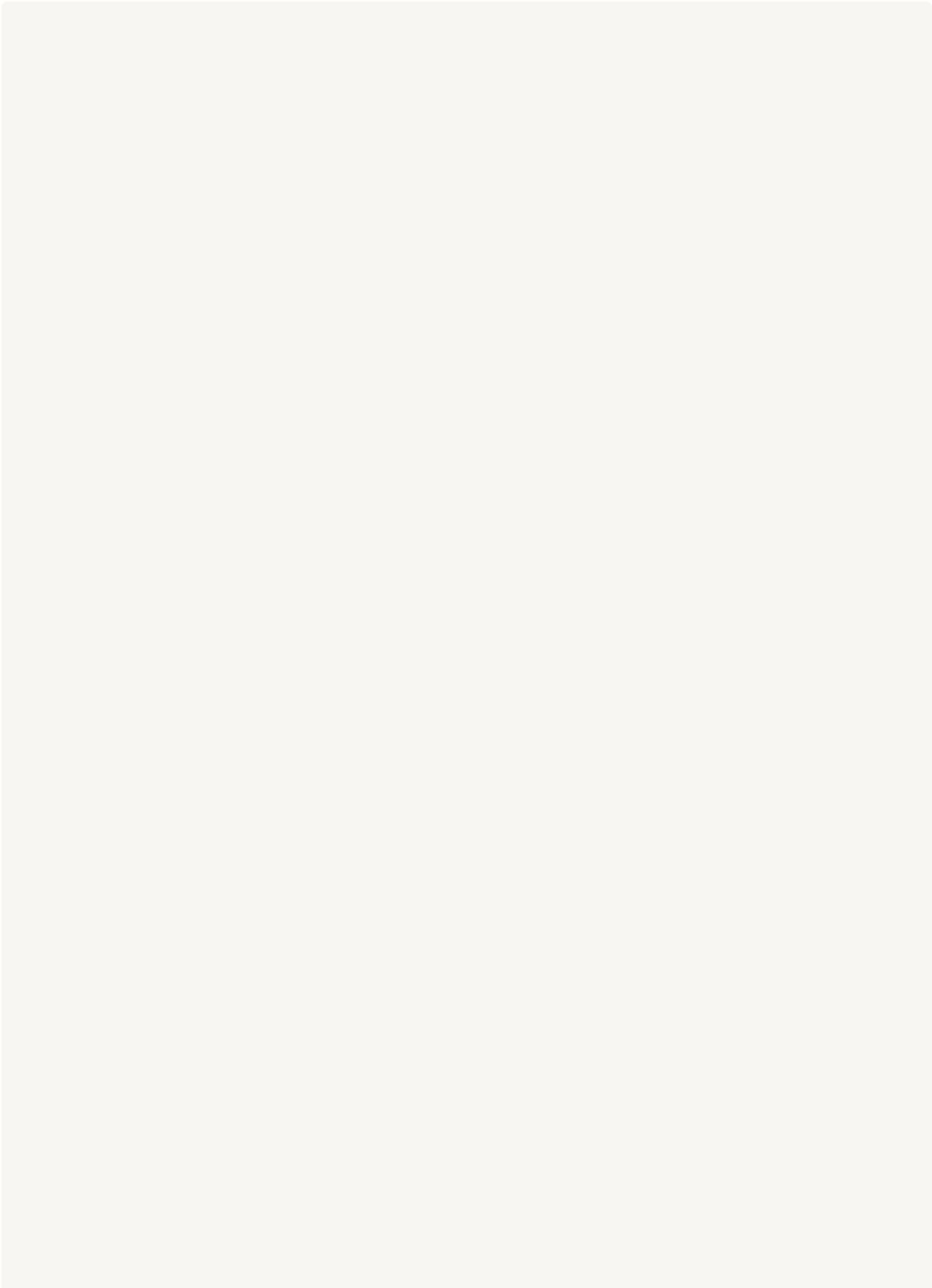
Run the existing offline (video-path → profile) pipeline as a realtime system on a single RTSP camera.

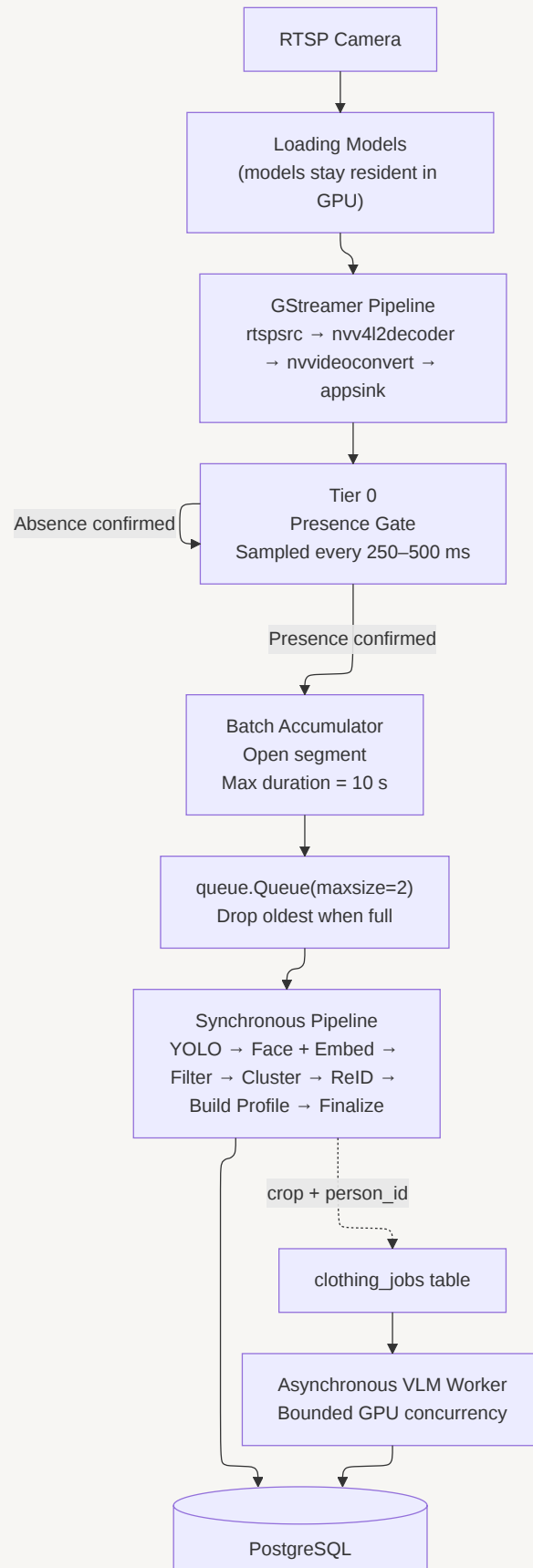
`rtsp://admin:Waycon2026@10.0.0104:554/Streaming/Channels/101`

On one NVIDIA RTX 5080 (16GB VRAM), with the full detection pipeline synchronous and only clothing-description (VLM) decoupled as async.

Architecture: presence-gated, bounded micro-batching

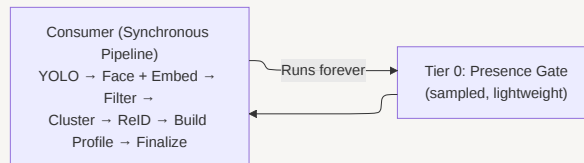
Main Processing Loop :





persons / segments /
camera_events

Inner Loop :



This is not frame-by-frame processing, and it's not a fixed always-on 10s timer either — it's event-triggered, bounded-duration micro-batches.

1. Ingestion (single process, GPU-resident, no HTTP)

GStreamer: rtspsrc (jitter buffer, RTCP, reorder)

→ rtpH264depay/rtpH265depay

→ nvv4l2decoder (GPU decode)

→ nvvideoconvert

→ appsink, (all in one GLib MainLoop thread.)

- Models (YOLO person detector, YOLOv8-Face, FaceNet, ReID, InternVL) load once at process startup and stay resident — no per-segment reload, no separate face_engine HTTP service (collapsed in-process since the 16GB card handles the full stack for one camera).

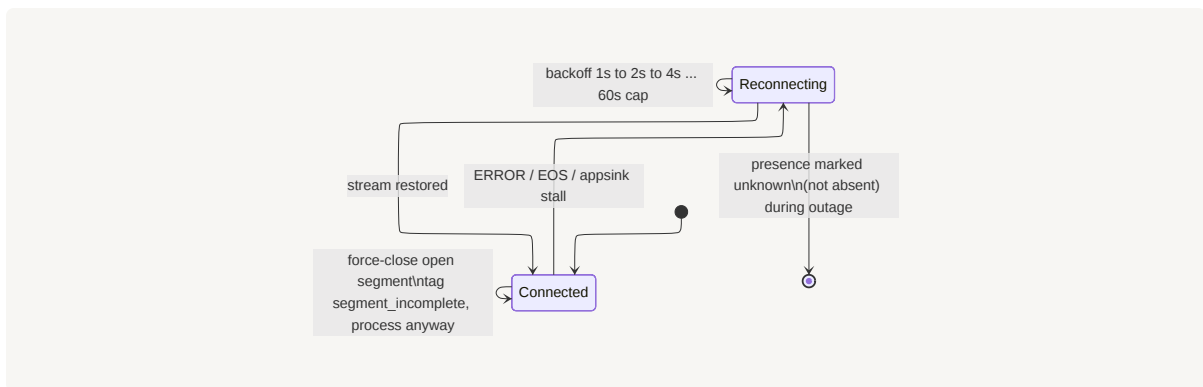
2. Presence-gated segmentation

- Tier 0 presence gate — cheap, sampled presence check (every 250–500ms, reused YOLO person detector, no downstream stages) runs continuously regardless of Tier 1 state, Open debounce: presence in ≥ 2 consecutive checks, Close debounce: **2–3s** grace period of absence.
- Tier 1 full pipeline — on debounced presence (≥ 2 consecutive positive checks), opens a SegmentBatch and runs the full sync pipeline: YOLO →

face detect+embed → quality filter → DBSCAN cluster → face/body pairing → ReID → build_profile → finalize.

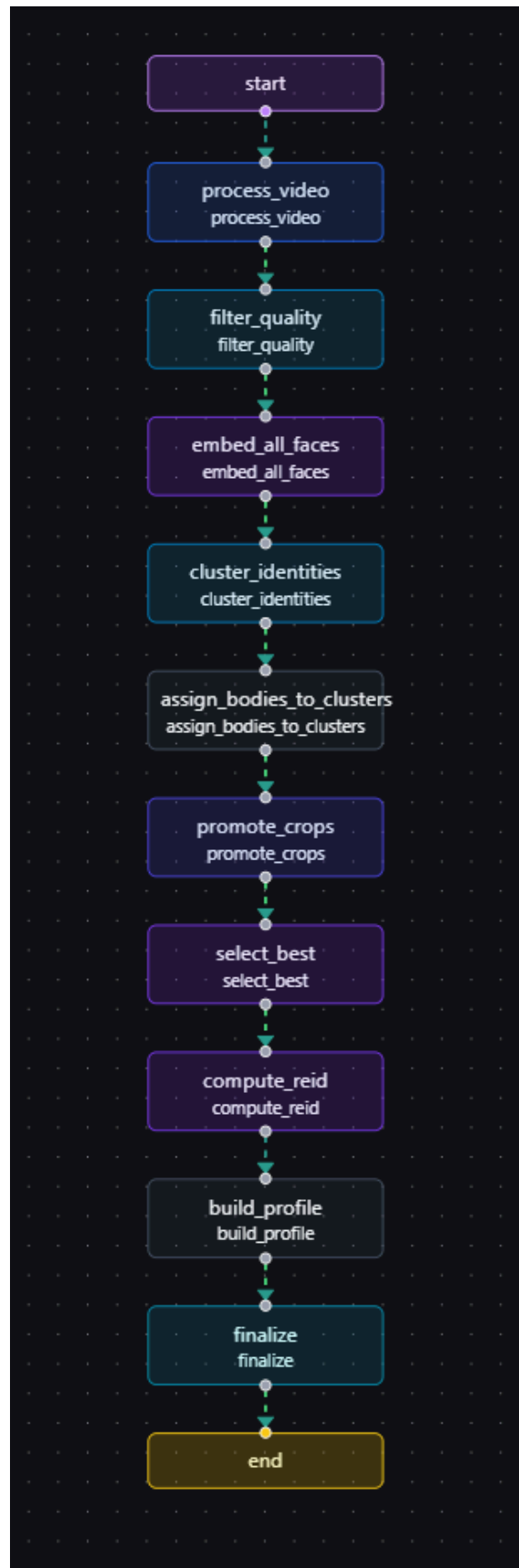
- While presence continues, segments still cap at 10s and rotate — bounded duration is preserved regardless of how long someone stays in frame.
- On absence, require a grace period (2–3s of continuous no-detection) before closing, to avoid fragmenting one visit into many segments from momentary misses.
- SegmentBatch: segment_start_ts, segment_end_ts, codec, frame tensor stack, per-frame timestamps, seq_num.

3. Reconnection & fault handling



- GStreamer bus (or appsink time-since-last-buffer watchdog) detects ERROR/EOS/stall → teardown, exponential backoff (1s → 2s → ... → 60s cap), rebuild pipeline.
- Any currently-open segment is force-closed, tagged segment_incomplete: true, and processed through the full pipeline as-is — never discarded.
- After reconnect, Tier 0's presence debounce always restarts from scratch — never assume presence carries across a lost-frame gap.
- Sequence numbers keep incrementing across reconnects (not reset), but since presence-gating already produces sparse, non-periodic segments, a sequence gap alone no longer proves a disconnect happened (idle periods look the same). A separate camera_events log (connected / disconnected / reconnect_attempt / reconnected_at) is the authoritative record of outages; during a reconnect attempt, presence state is reported as unknown, not absent.

1. LangGraph — same DAG (Directed Acyclic Graph), one node changed



keeps its shape. Two changes:

- `process_video/process_live_stream` are invoked once per `SegmentBatch` (in-memory tensors), not by opening a video file.
- `describe_clothing` is removed from the sync graph. `finalize` becomes the terminal node, and after `GlobalMemory.register()` resolves `person_id`, it:
 - a. persists the best body crop(s) to disk (durable, since GPU tensors won't outlive the batch),
 - b. inserts a row into `clothing_jobs` (`job_id`, `person_id`, `segment_id`, `crop_path`, `pipeline_version`, `status='pending'`).

No blocking call — just a fast insert.

2. Async VLM worker

- Separate long-running thread polling `clothing_jobs` for pending rows, runs InternVL, writes to appearances keyed by `person_id`, marks done (or `failed_retryable/failed_final` with backoff, same shape as the segment state machine).
- GPU contention bound: dedicated CUDA stream, hard cap of 1 in-flight VLM inference — prevents it from stealing cycles from the real-time detection path. VLM jobs aren't latency-sensitive, so a backlog draining later is fine.
- Idempotency: unique (`person_id`, `segment_id`) on both `clothing_jobs` and `appearances`.

Async VLM subsystem

Decoupled from the sync critical path at `finalize`, once `person_id` is resolved — not before.

Step	Actor	Action
1	<code>finalize (sync)</code>	persist best body crop to disk, resolve <code>person_id</code> via <code>GlobalMemory.register()</code>
2	<code>finalize (sync)</code>	insert <code>clothing_jobs</code> row — status pending, one fast INSERT, no blocking call

Step	Actor	Action
3	VLM worker (async)	poll pending rows, load crop, run InternVL on a dedicated CUDA stream, cap 1 in-flight
4	VLM worker (async)	write appearances keyed by person_id, mark done or retry → failed_final

3. Database: PostgreSQL

Replaces SQLite. Normal relational columns for frequently-queried fields, JSONB only for flexible model-specific metadata; time-range partitioning for large event tables once they grow.

Tables:

- persons, appearances, recognition_log, person_gallery, counters — unchanged shape from today.
- segments — segment_id, segment_start_ts, segment_end_ts, status, retry_count, segment_incomplete, seq_num.
- camera_events — event_type, ts, reason.
- clothing_jobs — as above.
- persons.embedding as vector(512) (pgvector) instead of a raw BLOB, so identity matching becomes an indexed similarity query instead of the current hand-rolled Python cosine scan in _find_existing_person — matters once lookups happen continuously in realtime rather than once per offline run.

Idempotency key: (segment_start_time, pipeline_version) — no camera_id needed, single camera.

4. Reliability state machine

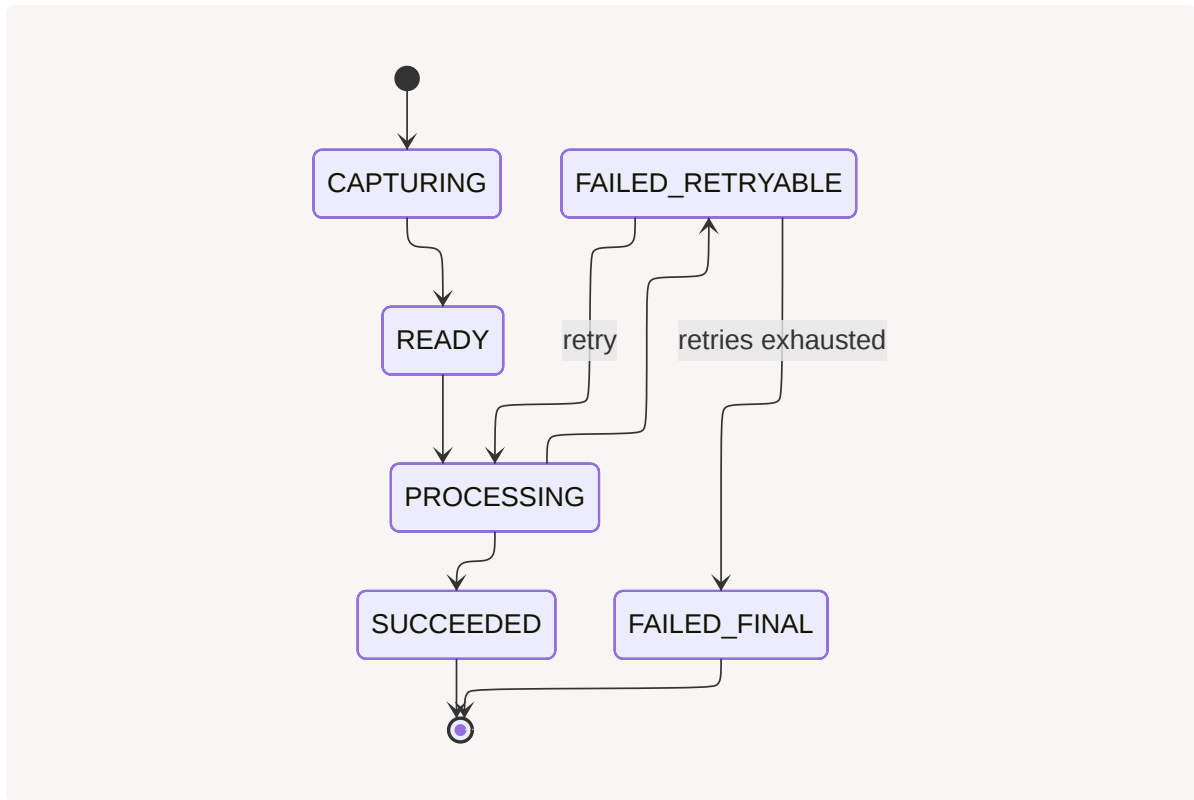
CAPTURING → READY → PROCESSING → SUCCEEDED

🔽 FAILED_RETRYABLE

🔽 FAILED_FINAL

Note: SUCCEEDED reflects core detection/identity completion only — clothing enrichment via clothing_jobs may still be pending/processing independently.

Don't conflate the two; a monitoring API should show both statuses separately.



Optimization priorities (unchanged, still in order)

1. Load all models once at process startup, never per-segment.
2. Instrument every stage (decode, inference, post-processing, DB, GPU util/mem) before optimizing anything.
3. Batch model inference within a segment; keep the orchestration unit as one segment.
4. Keep LangGraph state to references + compact metadata — avoid repeated NumPy/PIL/Torch conversions and intermediate image writes.
5. Once real bottlenecks are identified: try ONNX Runtime/TensorRT, torch.compile/inference mode, GPU-accelerated decode.
6. Keep GPU inference workers unblocked by CPU-bound work (archival, DB formatting) — run those on separate threads/processes.

Implementation :

Claude miss this implemenation :

What I'd actually do, if you want it now rather than deferred to when §3 collapses face_engine in-process:

- Pipeline: rtspsrc → rtph264depay → h264parse → nvh264dec → nvvideoconvert (NVMM out) → appsink (or nvv4l2decoder if you're actually deploying on Jetson, not the 5080 — worth confirming which).
- Use appsink with caps=video/x-raw(memory:NVMM) and map the buffer via pyDMABUF/pycuda/torch.cuda zero-copy (typically via Gst.Memory → cudaGraphicsGLRegisterBuffer/DMABUF import, or DeepStream's pyds/gst-nvdsbufferpool if you bring in DeepStream), instead of buf.map() into host memory.
- That changes BufferedFrame.frame from a NumPy array to a GPU tensor handle, which ripples into process_video.py/process_live_stream.py's crop-and-imwrite logic (those need a host copy anyway for the JPEG crops it saves to disk) — so it's not a localized fix, it's coupled to how §3 restructures the detection nodes to operate on GPU tensors natively.

Given that coupling, do you want me to do the zero-copy NVMM version now (I'll need you to confirm nvh264dec vs nvv4l2decoder for your actual deployment target and whether DeepStream is available/acceptable), or keep the current CPU-landing version until §3 redesigns the node interfaces around GPU-resident frames so we don't do this rework twice?

Steps :

1. Prerequisite cleanup

- a. Fix forensics/person_creation/graph.py — currently broken (missing node registrations for load_models/process_live_stream, dead describe_clothing import, orphaned builder.add_edge(...) calls after return, no START edge). Either revert to the last-good HEAD version or rebuild it correctly as part of task 5 below — pipeline cannot run at all right now.
- b. Restore the accidentally-deleted forensics/global_memory.db-x-persons-1-embedding.bin or confirm it's disposable debug output (git status shows

it as D, unstaged).

- c. Remove/resolve dead code found during the audit: `nodes/embed_faces.py` (superseded by `embed_all_faces.py+cluster_identities.py`, unreferenced in graph), `profile_signals.py::build_reid_signal` (imports an unused `models/body_reid.py`, superseded by `compute_reid.py`), and decide the fate of the legacy filesystem `profile.json/person_identifier.Config.PROFILE_ROOT` abstraction (parallel to `GlobalMemory`, used by `/api/profiles*` and `add_face_photos.py`) — either migrate it onto `GlobalMemory/Postgres` or deprecate those endpoints.

2. Ingestion: GStreamer replacing OpenCV

- a. Add GStreamer Python bindings (`PyGObject/gst-python`, `GStreamer + gst-plugins-good/bad/ugly`, `nv4l2decoder` from `DeepStream/L4T`) to the environment; nothing currently in `requirements-person-creation.txt` covers this.
- b. Replace `forensics/person_creation/live_stream.py`'s `LiveFrameBuffer` (`OpenCV cv2.VideoCapture + manual reader thread + drop-oldest queue.Queue`) with a GStreamer pipeline: `rtspsrc → rtph264depay/rtph265depay → nv4l2decoder → nvvideoconvert → appsink`, running in a `Glib MainLoop` thread. Keep the external interface (`get(timeout)`-style consumer, `stop()`) similar so downstream nodes change minimally.
- c. Implement the GStreamer `bus/appsink-stall watchdog → teardown → exponential backoff (1s → 2s → 4s...60s cap) → rebuild-pipeline reconnection logic` (state machine: `Connected ↔ Reconnecting`). Nothing like this exists today; current code has no `reconnect/backoff` at all.
- d. On reconnect, force-close any open segment, tag `segment_incomplete: true`, and push it through the pipeline anyway (never discard) — new logic, hook into task 10's `SegmentBatch`.
- e. Presence must report `unknown` (not `absent`) during a reconnect attempt, and Tier 0 debounce must restart from scratch post-reconnect — new state to track alongside presence gating.

3. Presence gating & segmentation (new subsystem, doesn't exist today)

- a. Build Tier 0 presence gate: sampled (250–500ms) cheap YOLO-person-only check, independent of Tier 1, with open debounce (≥ 2 consecutive positive checks) and close debounce (2–3s grace period).
 - b. Build SegmentBatch accumulator: opens on debounced presence, caps duration at 10s (rotates while presence continues), holds segment_start_ts, segment_end_ts, codec, frame tensor stack, per-frame timestamps, seq_num. Doesn't exist — today a "segment" is an entire video file or a fixed duration_seconds live-stream window (process_live_stream.py), not presence-triggered.
 - c. Wire Tier 0 → queue.Queue(maxsize=2) with drop-oldest-on-full semantics feeding the sync pipeline (Tier 1) — replaces the current single LiveFrameBuffer queue used directly by process_live_stream.py.
4. Collapse face_engine into the in-process pipeline
- a. Eliminate the face_engine HTTP microservice as a separate process for the realtime path: load YOLO-Face + FaceNet directly in-process at startup (same singleton pattern face_engine/app.py already uses via get_face_detector()/get_face_embedder(), just imported directly instead of through FaceEngineClient).
 - b. Update every call site currently using FaceEngineClient() over HTTP — load_models.py, process_video.py, process_live_stream.py (→ its GStreamer replacement), embed_all_faces.py, auto_pair.py, tools/add_face_photos.py — to call the in-process detector/embedder directly, removing the per-crop JPEG-encode/HTTP-round-trip overhead.
 - c. Deduplicate the redundant face-embedding computation: today the same crop can be embedded once in embed_all_faces.py and again inside auto_pair.py's cue scoring — cache/reuse the embedding instead of recomputing.
 - d. Decide what happens to face_engine's /recognize route, which currently talks to GlobalMemory/SQLite directly from inside the "microservice" — either retire it for the realtime path or keep face_engine alive only for non-realtime/offline enrollment use cases (add_face_photos.py), now pointed at Postgres.

- e. Models loaded once at process startup, resident for the process lifetime, no per-segment reload — audit `load_models.py` and the GStreamer-based main loop to ensure this holds (currently true per-run, but "per-run" today means per-HTTP-request-triggered pipeline execution via `service.py`'s per-request thread spawn, which needs to become one long-lived process).

5. LangGraph changes

- a. Rebuild `graph.py`'s DAG (task 1) so `process_video/process_live_stream` are invoked once per SegmentBatch using in-memory frame tensors, not by opening a file path — this changes `process_video.py`'s core assumption (`cv2.VideoCapture(video_path)` file-open loop) and `process_live_stream.py` entirely.
- b. Remove `describe_clothing` from the synchronous graph; make `finalize` the terminal node.
- c. In `finalize.py`, after `GlobalMemory.register()` resolves `person_id`: (a) persist best body crop(s) to disk, (b) insert a `clothing_jobs` row (`job_id`, `person_id`, `segment_id`, `crop_path`, `pipeline_version`, `status='pending'`) as a fast non-blocking INSERT — replaces today's synchronous `describe_clothing.py` VLM call inside the graph.
- d. Revisit `state.py`'s `PersonCreationState` for the realtime path: it currently stores embeddings as JSON-serializable `list[float]` and crops as file paths (already compatible with the "references + compact metadata" goal), but the `_status_callback: Callable` field is not serializable and was presumably why the `MemorySaver` checkpointer was removed in 3cb5a3b2 — if any checkpointing/resumability is wanted for segment recovery, this field needs to be pulled out of state.

6. Async VLM worker

- a. Build a standalone long-running worker thread/process polling `clothing_jobs` for `status='pending'`, loading the crop, running InternVL (`get_clothing_describer()`, already used in `describe_clothing.py` — reuse that model-loading code) on a dedicated CUDA stream, hard-capped at 1 in-flight inference.

- b. Write results into appearances keyed by person_id; mark done or failed_retryable/failed_final with backoff (mirror the segment reliability state machine).
- c. Enforce idempotency via UNIQUE(person_id, segment_id) on both clothing_jobs and appearances (schema change, see task 27).
- d. Reliability state machine
- e. Implement CAPTURING → READY → PROCESSING → SUCCEEDED / FAILED_RETRYABLE ⇌ / FAILED_FINAL for segments — no equivalent exists today (current jobs are just queued/running/succeeded/failed in service.py's in-memory _jobs dict, not persisted, lost on restart).
- f. Expose monitoring that separately shows core-detection-SUCCEEDED vs. clothing-enrichment status per person/segment (don't conflate, per your note) — new API surface, since service.py's current status endpoint reflects one LangGraph run's node progress, not segment/person-level state.

7. PostgreSQL migration

- a. Add psycopg[binary] + pgvector to dependencies; stand up a Postgres instance (no Dockerfile/compose exists in the repo at all — will need one, or point at an existing instance).
- b. Port forensics/global_memory/schema.sql to Postgres DDL: keep persons/appearances/recognition_log/person_gallery/counters shape, add segments (segment_id, segment_start_ts, segment_end_ts, status, retry_count, segment_incomplete, seq_num), camera_events (event_type, ts, reason), clothing_jobs. Use normal columns for queried fields, JSONB only for flexible metadata (today several fields like cameras, best_body_crops, video_sources are JSON-as-TEXT — convert to JSONB). Add time-range partitioning for recognition_log/camera_events once volume justifies it.
- c. Change persons.embedding from raw BLOB to vector(512) (pgvector), and rewrite store.py::_find_existing_person/query_by_face (currently: fetch every row, stack into a NumPy matrix, dot-product in Python) to an

indexed $\Leftrightarrow/\Leftrightarrow$ similarity query with an ivfflat/hnsw index — this is the biggest correctness/perf rewrite in store.py.

- d. Rewrite GlobalMemory's locking model: today correctness relies on a single SQLite file + Python threading.RLock() + BEGIN IMMEDIATE. Postgres with a realtime worker (and possibly the async VLM worker) writing concurrently needs real transactions/row-level locking and a connection pool (e.g. psycopg_pool) instead.
- e. Set idempotency key (segment_start_time, pipeline_version) on segments (no camera_id needed for single-camera scope).
- f. Update forensics/global_memory/config.py (currently only DB_PATH/SIMILARITY_THRESHOLD for SQLite) to Postgres connection config (host/port/db/user/password or DSN, pool size).
- g. Migrate/backfill existing SQLite data (forensics/global_memory.db) into Postgres if historical persons need to carry over.

8. Service layer

- a. Replace service.py's per-request threading.Thread pipeline spawn + in-memory _jobs dict (lost on restart, no locking around concurrent read/write) with the long-lived single realtime process model described in the architecture — /api/person/start as currently designed (one-shot video-file/camera job) doesn't map onto "one resident process continuously consuming one camera." Decide whether service.py becomes purely a monitoring/query API in front of Postgres (segments, camera_events, clothing_jobs status) rather than a job-launcher.
- b. Fix the unsandboxed GET /api/images?path= path-traversal issue while touching this file.
- c. Decide fate of offline video-file ingestion (process_video.py//api/person/start with video_paths) — architecture is camera-first, but offline enrollment is still referenced in docs (PERSON_CREATION_DOC.md) as a prerequisite step; likely keep both entry points but only camera path goes through the new segment/presence machinery.

9. Instrumentation & optimization (do after the above, per your stated priority order)
 - a. Instrument decode, inference (per model), post-processing, DB writes, and GPU util/mem before any optimization work — nothing like this exists in the codebase currently (no metrics/tracing found anywhere).
 - b. Centralize device/CUDA config (currently ad-hoc device="auto" scattered per-model-load call) so GPU residency and the VLM's dedicated CUDA stream are configured in one place.
 - c. Only after instrumentation: evaluate ONNX Runtime/TensorRT, torch.compile/inference mode, GPU-accelerated decode (GStreamer's nvv4l2decoder already covers decode).
 - d. Move CPU-bound work (crop archival, DB formatting/JSON writes — currently done inline in finalize.py, promote_crops.py) off the GPU-inference thread onto separate threads/processes.

Postgresql migration :

Design choices

1. GlobalMemory's public API stays 100% identical. Every call site (finalize.py, service.py, vlm_worker.py, face_engine/local_client.py, face_engine/routes/recognize.py) keeps calling `gm.register(...)`, `gm.query_by_face(...)`, `gm.upsert_segment(...)`, etc. exactly as today. This is a backend swap, not an interface change — zero edits needed outside global_memory/.
2. Full cutover to Postgres, not a dual SQLite/Postgres backend. The architecture doc says Postgres "replaces SQLite," and maintaining both means two SQL dialects, two locking models, two things to test. The SQLite file becomes a one-time backfill source (via a migration script), not a fallback. (Open question below.)
3. Timestamps: TEXT (ISO8601 strings) → TIMESTAMPTZ. Needed for correct time-range partitioning later and is the idiomatic Postgres type; `psycopg3` converts Python datetime ↔ TIMESTAMPTZ automatically, so store.py's logic barely changes.

4. `persons.embedding`: BLOB → vector(512) with an HNSW index, `vector_cosine_ops`. Embeddings are already L2-normalized, so cosine distance ($\Leftrightarrow$) is equivalent to today's dot-product similarity — same math, now index-accelerated instead of a full-table NumPy scan. I'm picking HNSW over IVFFlat: no need to retrain/rebuild as the table grows, better recall, and IVFFlat's "pick a lists parameter for your expected row count" tuning doesn't fit a system that starts at zero rows and grows continuously.
5. Locking: `psycopg2.pool.ConnectionPool` + a Postgres advisory lock (`pg_advisory_xact_lock`) scoped around `register()`'s find-or-create critical section, replacing the Python `threading.RLock()` + SQLite `BEGIN IMMEDIATE`. Row-level `SELECT ... FOR UPDATE` doesn't fit here because the "find existing" step is a similarity search, not a lookup by key — there's no row to lock until after you've already decided whether one matches. An advisory lock serializes the whole read-decide-write sequence, which is the same conceptual guarantee the current code provides, just correct across multiple connections/processes instead of one.
6. segments idempotency: add a `pipeline_version` column (doesn't exist in the current SQLite segments table) + `UNIQUE(segment_start_ts, pipeline_version)`. Keep `segment_id` (UUID) as the surrogate primary key so `clothing_jobs/camera_events` FKs stay simple, but the idempotency semantics you asked for live on the natural key, so a crash-recovery replay of the same capture window collapses correctly even though it'd generate a new UUID.
7. New `camera_events` table (`id, event_type, ts, reason`) per the original architecture doc. I'm adding the table now but not wiring `gst_stream.py`'s reconnect callbacks to write into it yet — that's a separate piece of scope (coupling the ingestion layer to the DB layer) I'd rather do as a deliberate follow-up than sneak in here.
8. JSONB for flexible fields: `cameras`, `best_body_crops`, `video_sources`, and the color-signal samples list — everything that's currently JSON-serialized into a TEXT column.
9. Partitioning: designed for, not applied. The task says "once volume justifies it" — I'll use proper `TIMESTAMPTZ` columns (a prerequisite for partitioning later) but won't declare `recognition_log/camera_events` as partitioned tables now. Converting a normal table to partitioned later is a bit of work but not a rewrite;

doing it prematurely for a single-camera deployment that won't hit real volume soon is needless complexity now.

10. Dependencies: `psycopg[binary,pool]` (`psycopg3`) + `pgvector` (Python package for the adapter) — not `psycopg2`. `psycopg3` has native connection-pool support (`psycopg_pool`) and is the actively developed driver.

Correction after the implementation :

`GstFrameBuffer`

↓ frames

`PresenceGatedIngestion`

└─ Tier 0 presence checks

└─ `SegmentAccumulator`

↓ closed `SegmentBatch`

`queue.Queue(maxsize=2)`

↓

`RealtimeProcessor.run_forever()`

↓

`process_segment()`

↓

`LangGraph` pipeline